

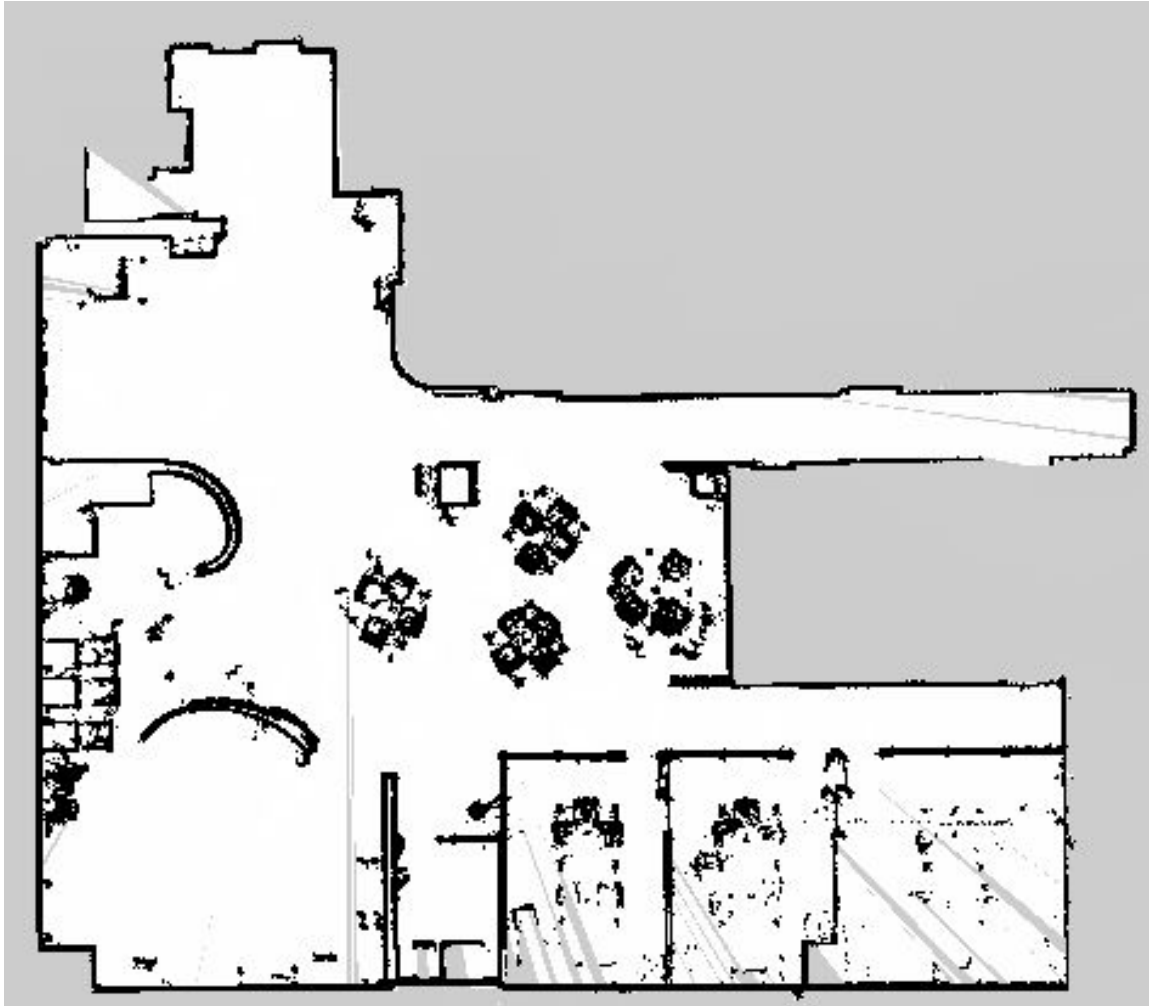
Multi-Robot Path Planning

Visibility graph generation

Introduction

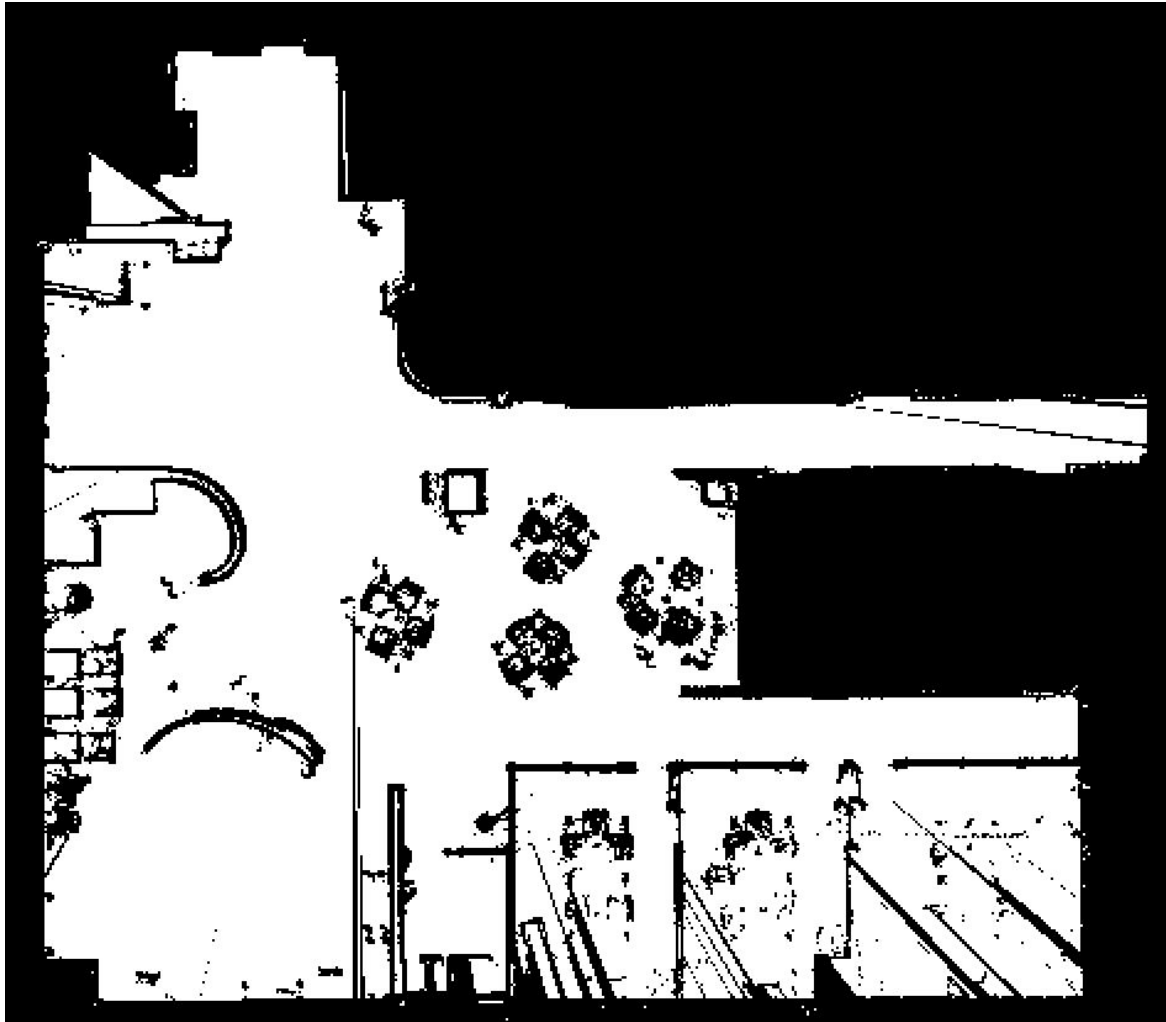
- This implementation basically implemented using python with the help of several libraries.
 - OpenCV: for basic image processing techniques
 - pyclipper: for calculate points of inset and offset polygons
 - numpy: for matrix calculations
- Next couple of slides explain the process in several steps

Step 1: Loading image and resizing



- Raw image can be .pgm or other image format
- Doubled the size of the image (but it depends on the original size)

Step 2: Binarization



- Binarize the image to set obstacle and unknown pixels to zero (black) while pixels represent free space set to 255 (white)
- In this case, pixel values are as follows:
 - 0 - obstacles
 - 205,204 - unknown
 - 254,255 - free space
- Threshold was set to 210.

Step 3: Erosion



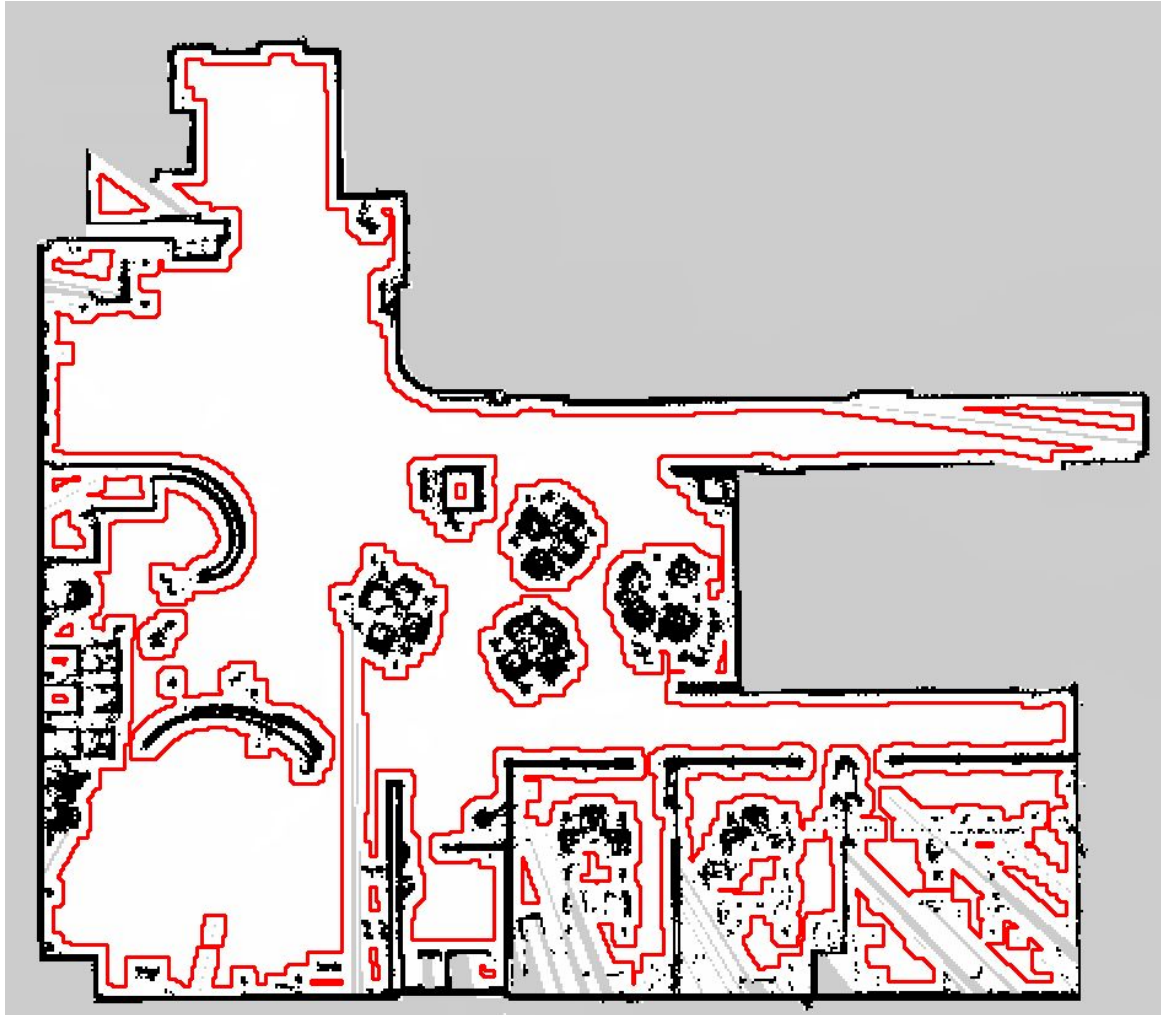
- Erode function in OpenCV has been performed to reduce the noise in the image.
- In this case, 3x3 kernel with 7 iterations was used.

Step 4: Contour detection



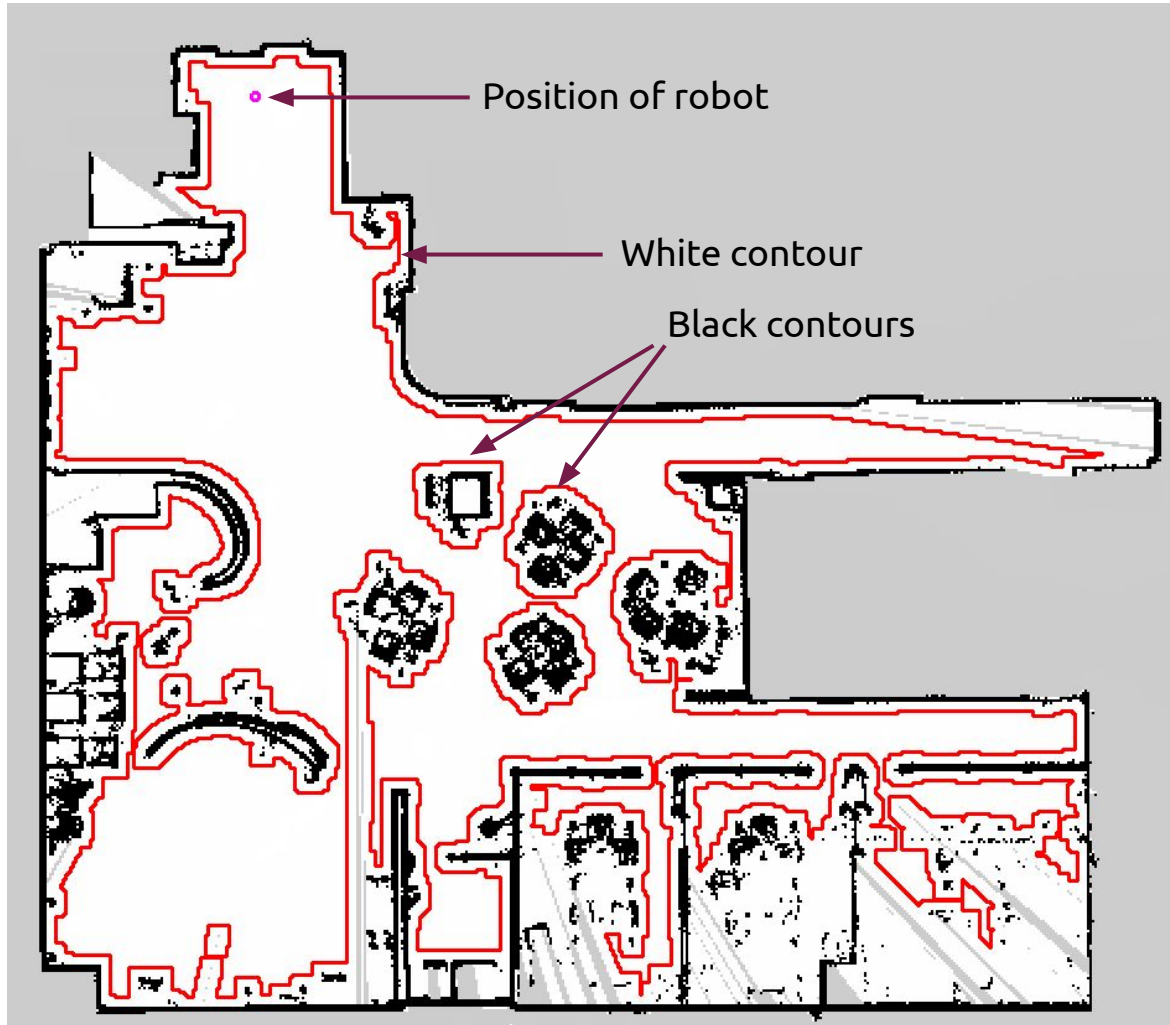
- All the contours were detected using OpenCV function.
- All the detected (both black and white) contours were highlighted in red.

Step 4: Contour detection



- Detected contours were represented in the original image.

Step 5: Contour selection



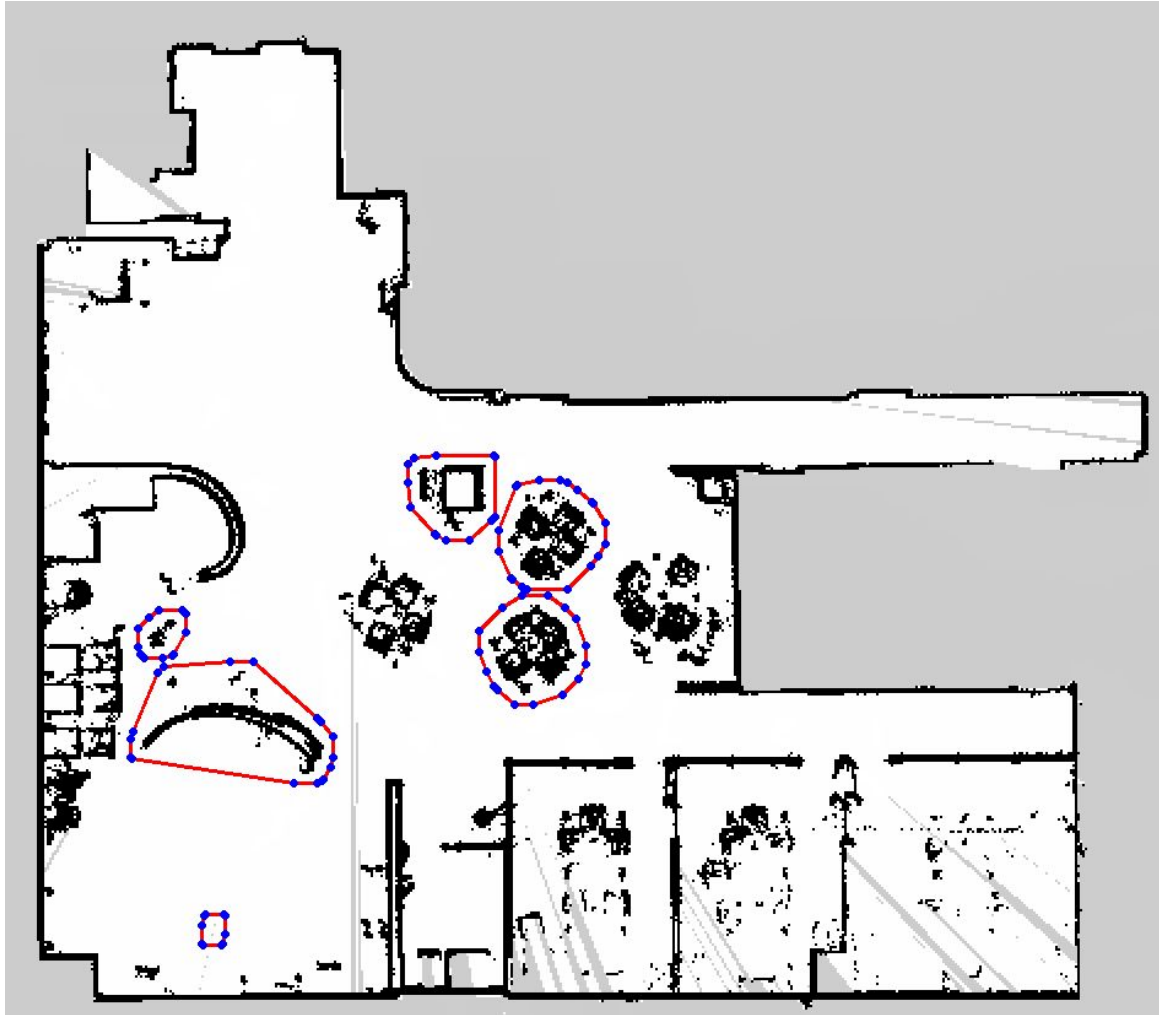
- White contour was selected according to the given position of robot (pink circle represents the position of robot).
- The black contours inside the selected white contour were also selected.

Step 6: Polygon approximation (white)



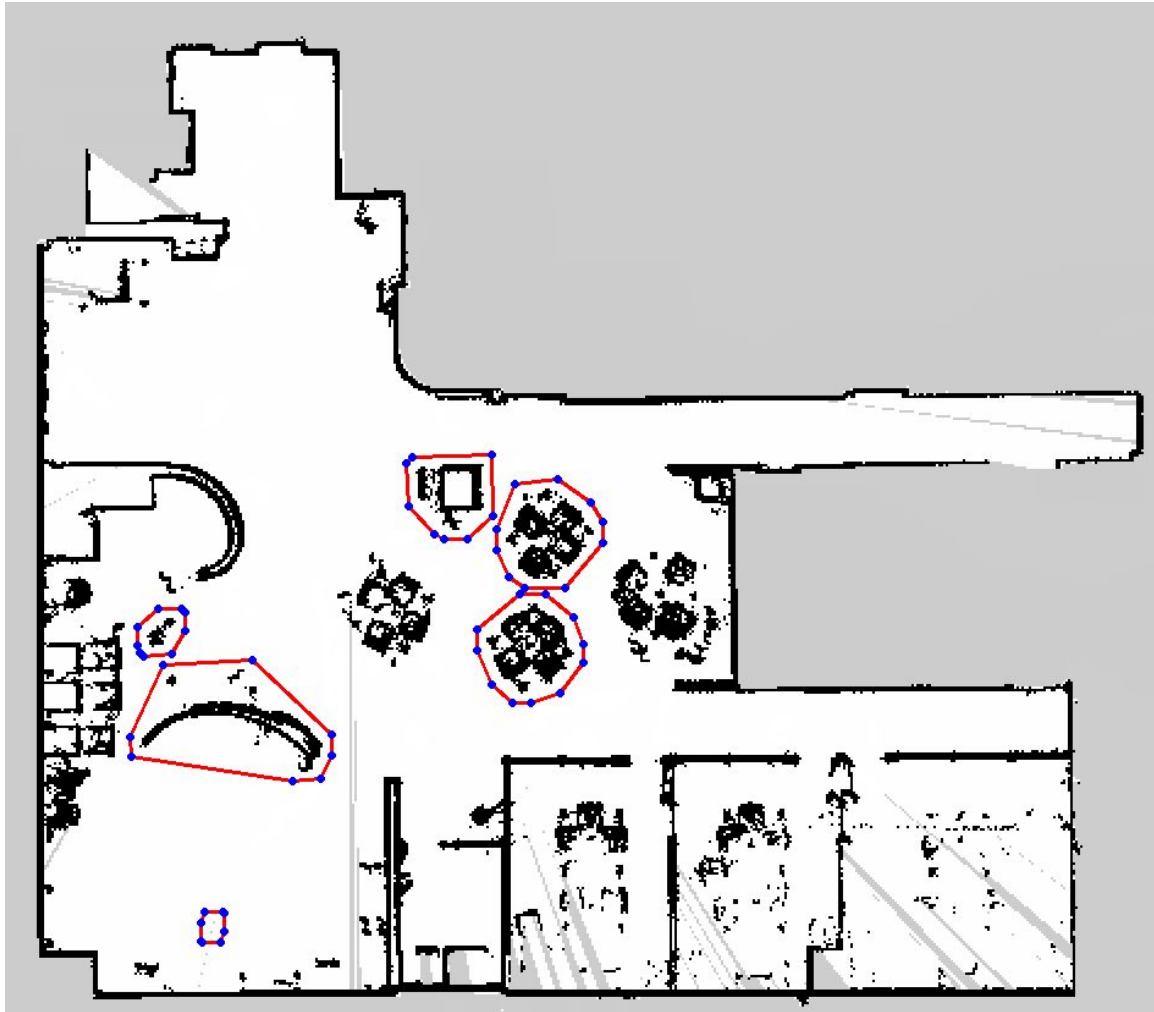
- Polygon approximation function (OpenCV) was performed to reduce the number of points in the outline of white contour.

Step 7: Convex hull (black)



- Convex hull was obtained for each black contour.

Step 8: Polygon approximation (black)



- To further reduce the number of points of black contour outline, polygon approximation was applied.

Step 8: Contour selection final output



- Blue dots represent all the necessary points at the end of the contour selection process.

Step 9: Inset polygons generation



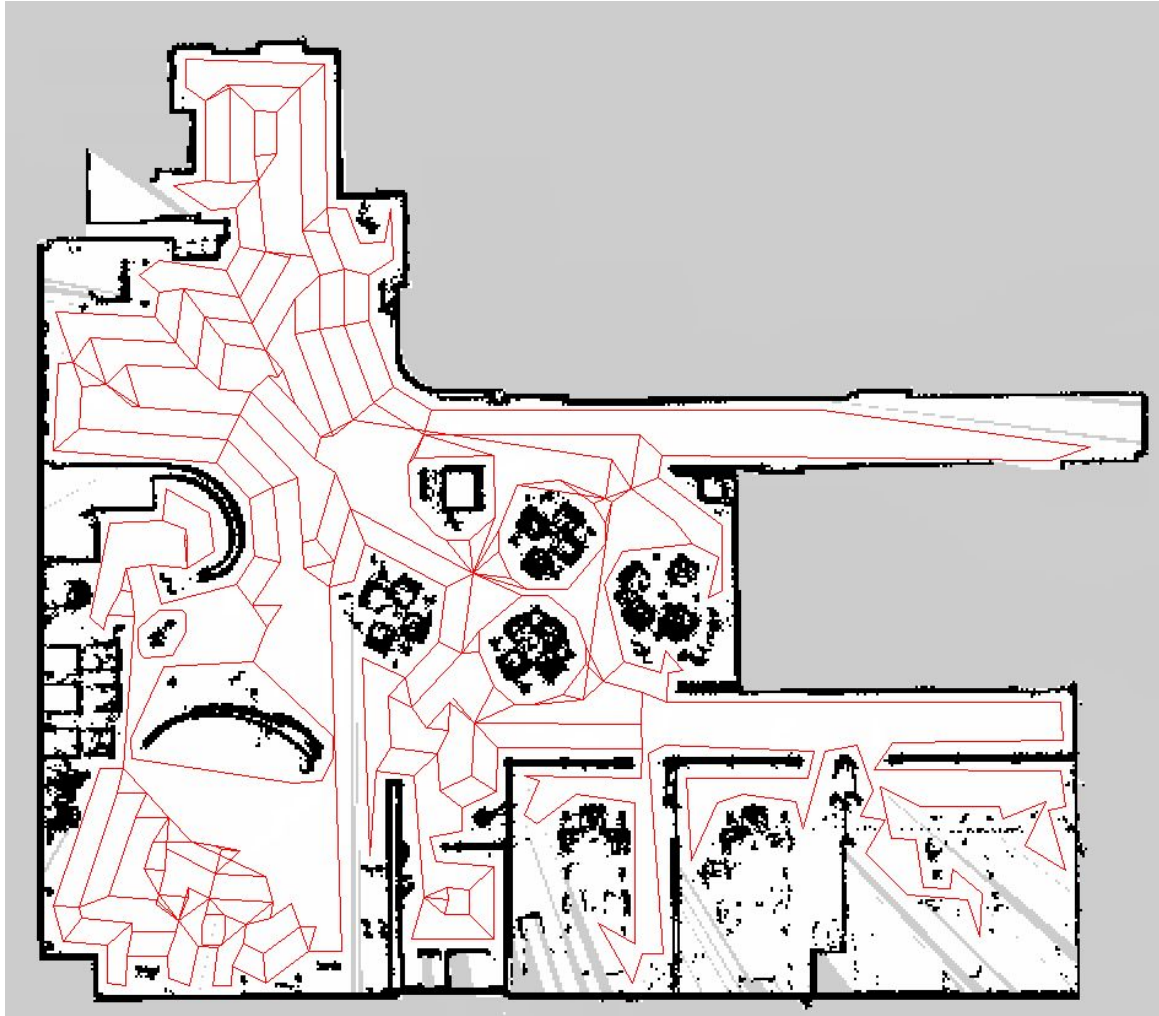
- Total number inset polygons possible were generated according to the given distance from the main white polygon (using pyclipper library).
- Green lines represent generated inset polygons.

Step 10: Removal of points



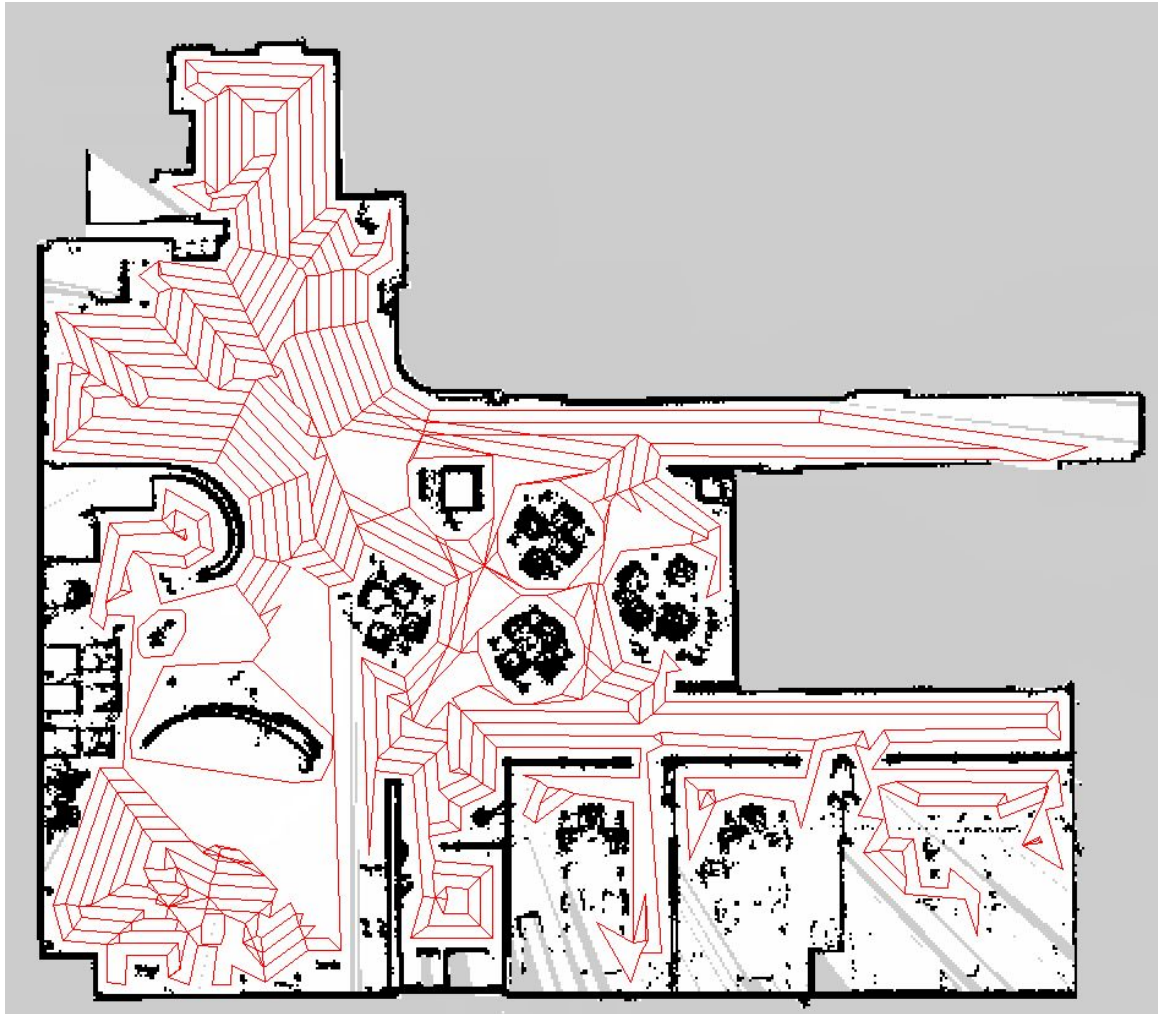
- Points of inset polygons that are inside the black polygons were removed.

Step 11: Make connection



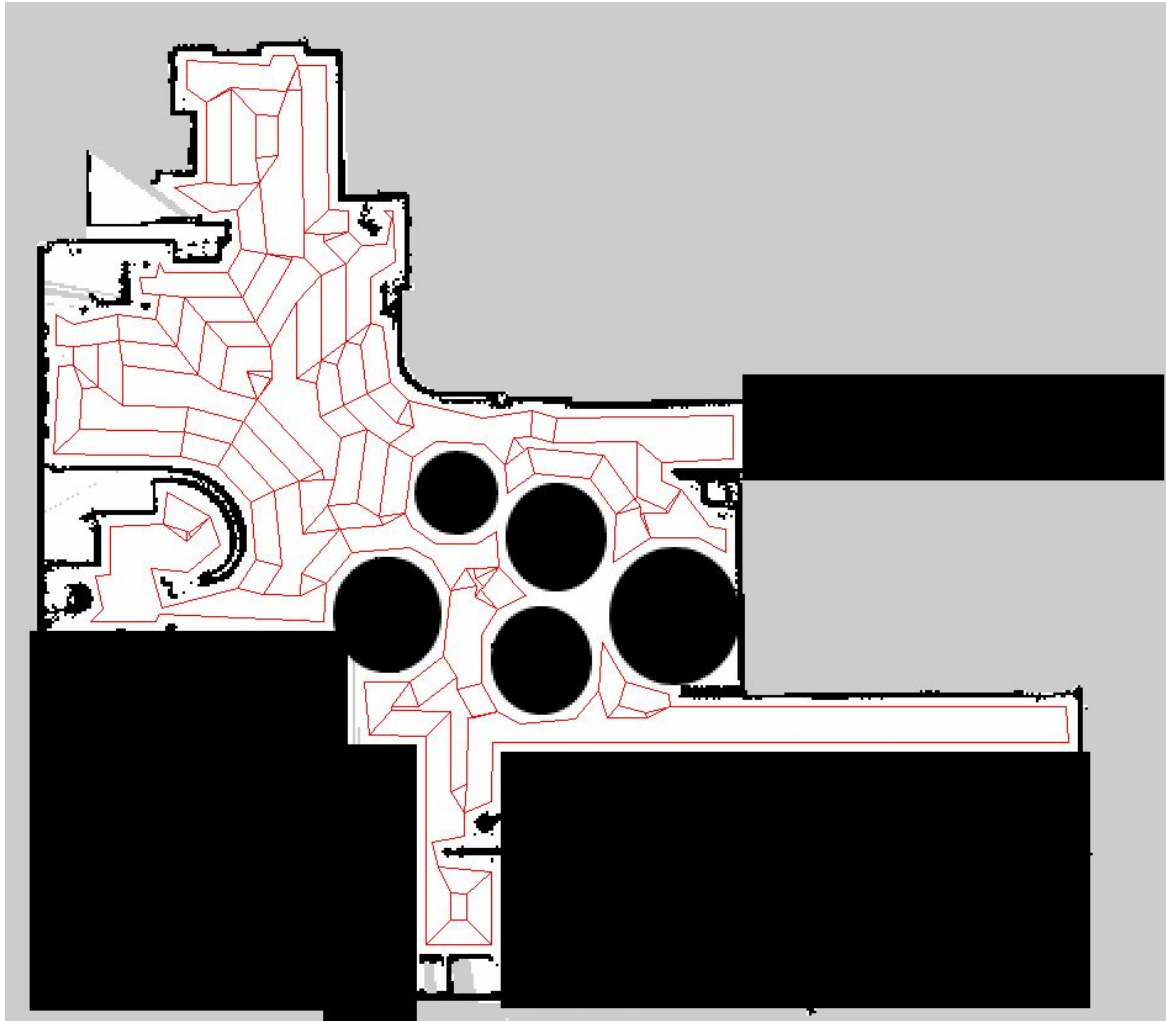
- Points of inner polygons were connected to closest point of outer polygon.
- All the connections were checked whether they intersect with black polygon or most outer white polygon.
- Finally, a list of points for possible connections for each point were generated as visibility graph.

Final result of graph generation



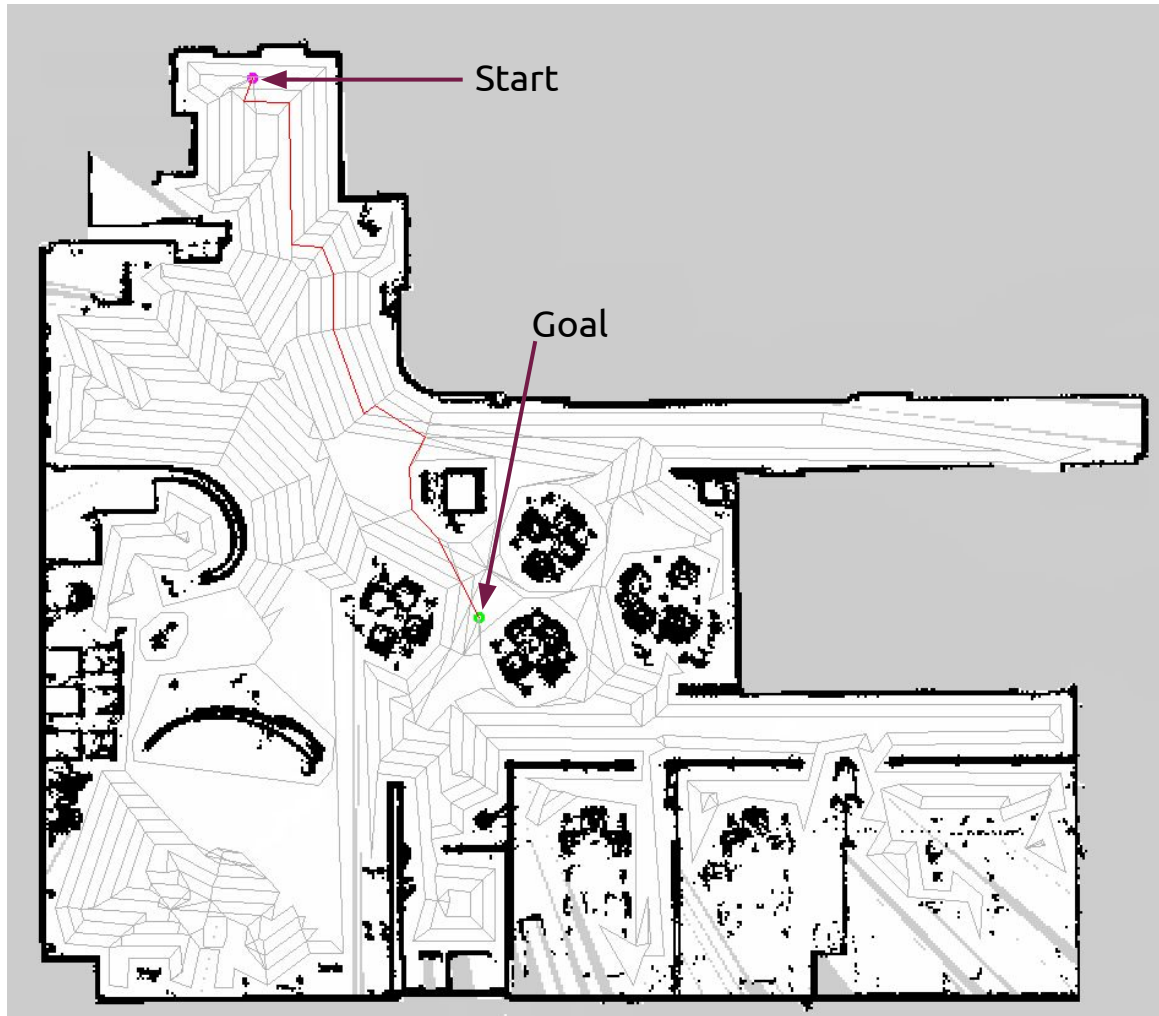
- This image shows the generated visibility graph with shorter distance between inset polygons.

Final result of graph generation



- Visibility graph generated by the implemented code for edited image.

Shortest path: test case 1



- First, the closest vertices in visibility graph were calculated for give start and goal points.
- Then, generated visibility graph was updated by adding start and goal point.
- Finally, shortest path was calculated using A* algorithm.

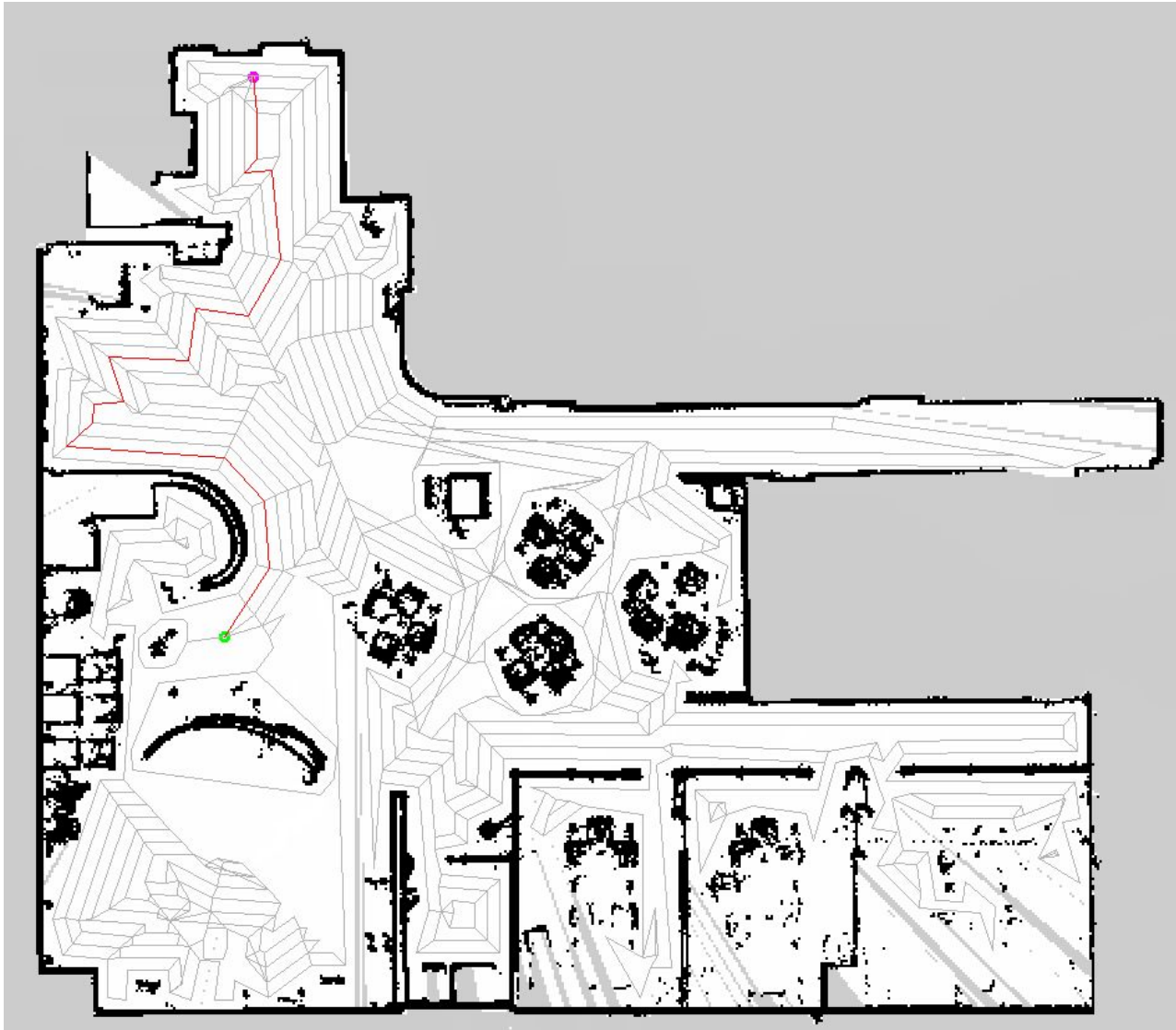
Shortest path: test case 2



Shortest path: test case 3



Shortest path: test case 4



Shortest path approximation: test case 1



- Approximation algorithm was used to remove unwanted points from the shortest path.
- Around 50% of points were removed from the calculated path.

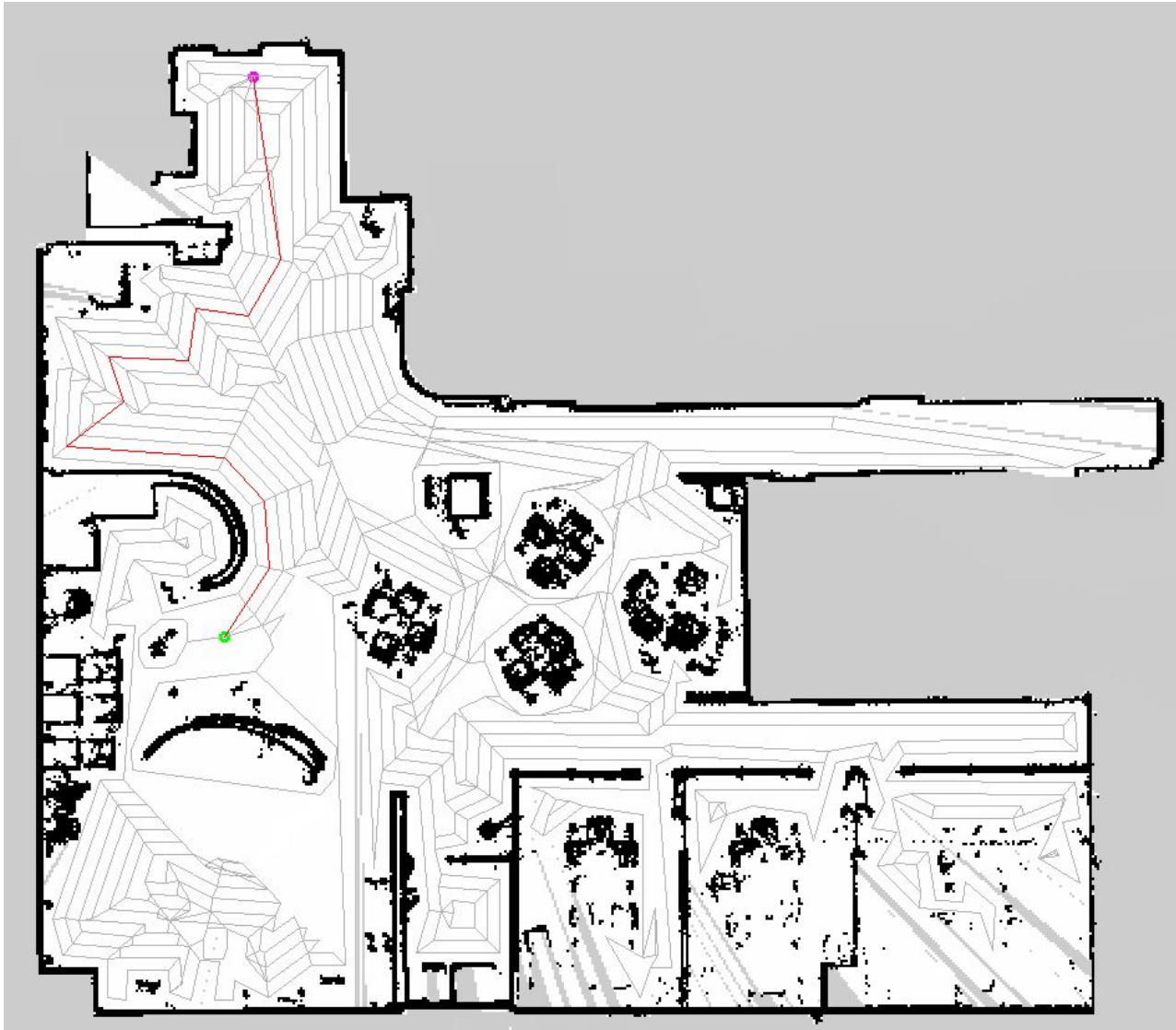
Shortest path approximation: test case 2



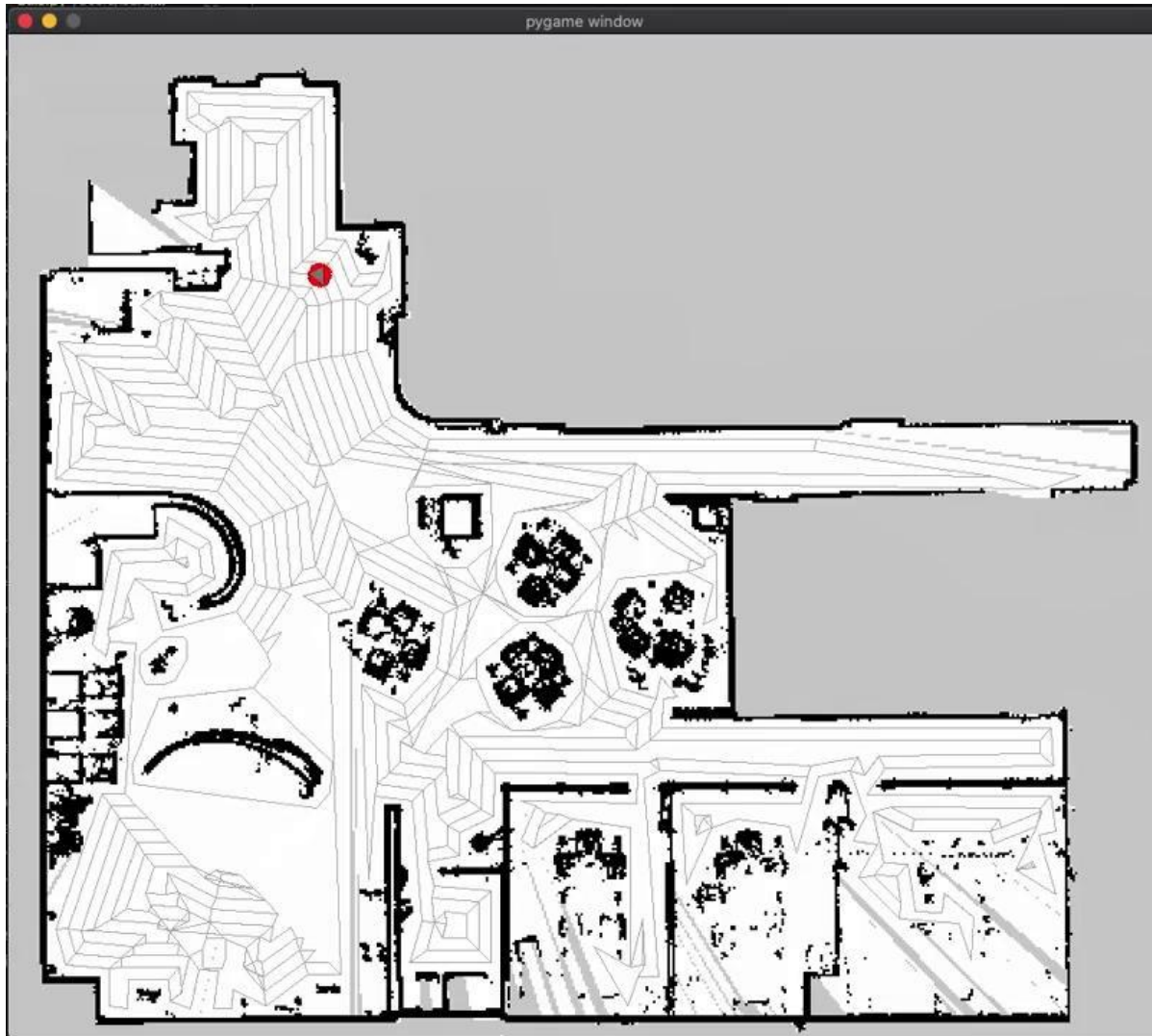
Shortest path approximation: test case 3



Shortest path approximation: test case 4



Robot navigation simulator



- Robot navigation simulator has been developed using pygame library.
- Simulator shows the navigation of the robot for the given shortest path.